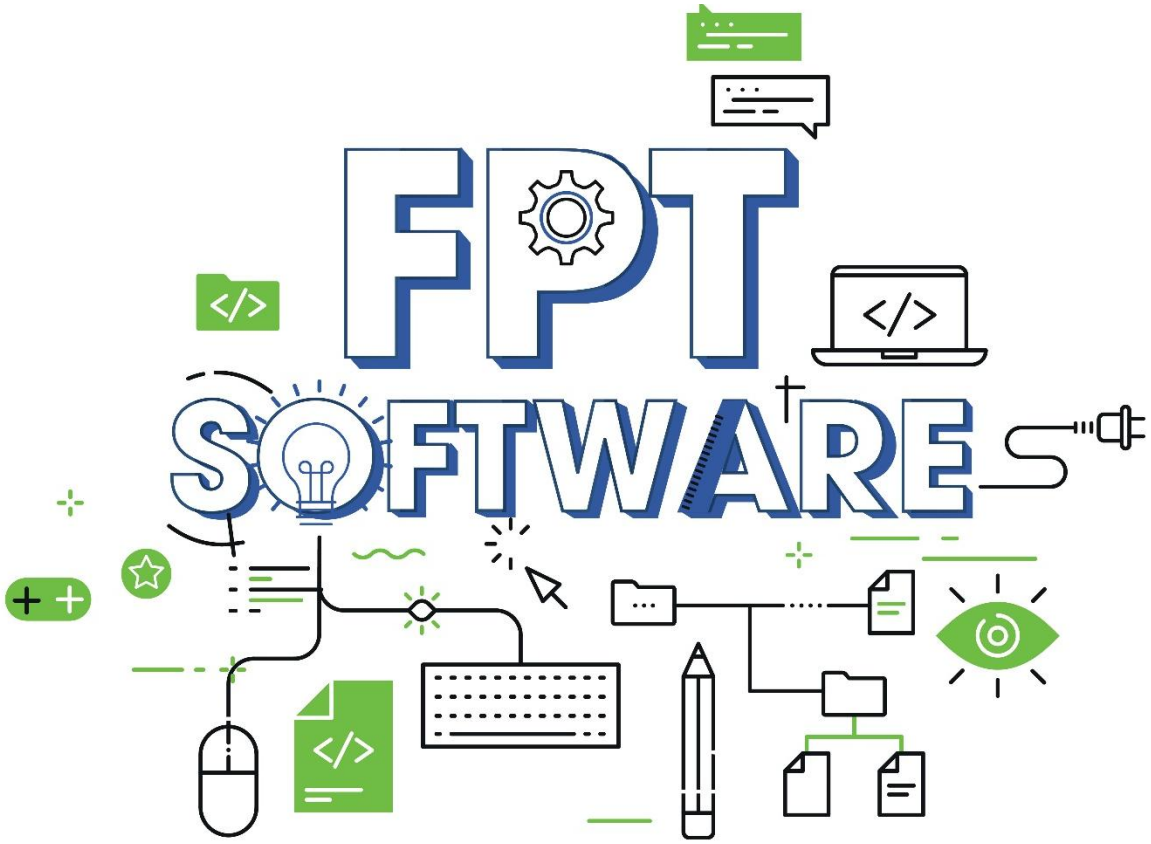
Explore Reactive Programming

DuyTD13



Content

- Overview Reactive programming
- *RxJava*
- *RxKotlin*
- *Common operator usuage*

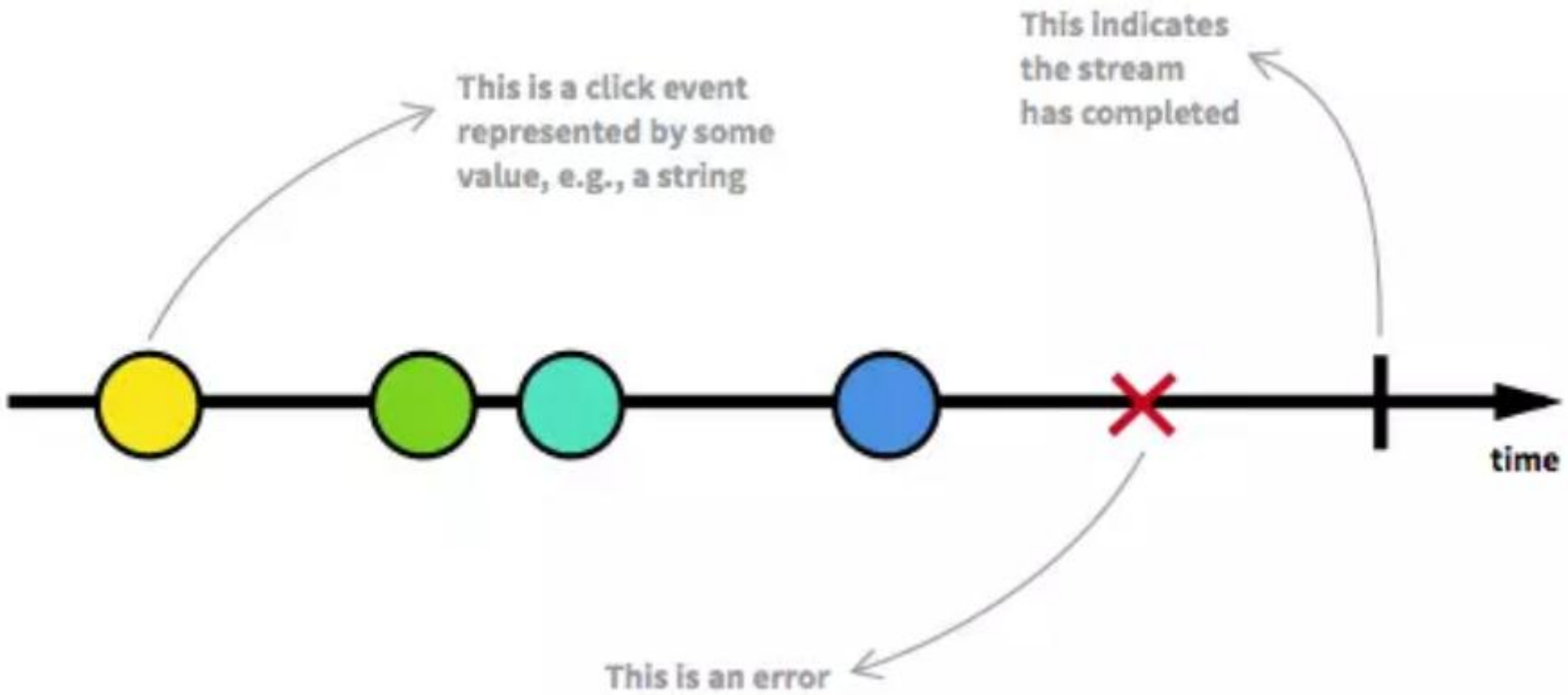
Section 1

Overview

Overview

- Asynchronous processing means that the processing of an event does not block the processing of other events
- **Reactive programming** is a declarative programming paradigm that is based on the idea of asynchronous event processing and data streams
- The strength of RP lies in applying functional programming, which allows filtering (filter, take, scan, ...), transforming from one stream to another (map, flatMap, reduce), or merging multiple streams into a new stream (combine, merge, zip, ...) quite easily without altering the state of the original stream

Overview


This is a click event
represented by some
value, e.g., a string
This indicates
the stream
has completed
time
This is an error

Overview

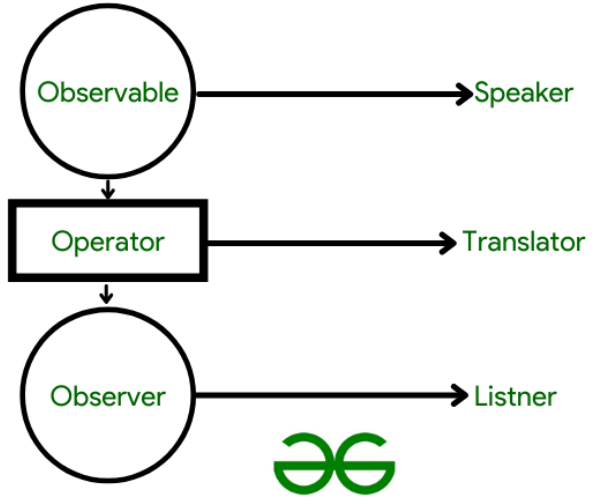
- Reactive Extension (ReactiveX or RX) is a library that follows the principles of Reactive Programming, which means it composes asynchronous programs based on events using observable sequences.
- These libraries provide a list of interfaces and methods that help developers write code in a simpler and more concise manner.
- Rx is a combination of the best ideas from the Observer pattern, Iterator pattern, and functional programming.
- Reactive Extension is available in many languages such as C++ (RxCpp), C# (Rx.NET), Java (RxJava), Kotlin (RxKotlin), Swift (RxSwift), ...

Section 2

RxJava

RxJava

RxJava is fundamentally a library that provides asynchronous events developed based on the Observer Pattern. You can create asynchronous data streams on any thread, manipulate data, and use it with Observers. The RxJava library offers various excellent operators such as map, combine, merge, filter, and many others that can be applied to data streams



RxJava

- Fundamentally, RxJava has two main components: Observable and Observer. In addition, there are other elements such as Schedulers, Operators, and Subscription, which play roles like multithreading, data manipulation, and connection. Let's get acquainted with each component:
- **Observable**: It is a data stream that performs some tasks and emits data.
- **Observer**: It is an essential accompanying component of Observable. It receives the data emitted by Observable.
- **Subscription**: It is the link between Observable and Observer. Multiple Observers can subscribe to a single Observable.
- **Operator**: It supports modifying the data emitted by Observable before the Observer receives them.
- **Schedulers**: Schedulers determine the thread on which Observable will emit data and on which thread Observer will receive data

RxJava - Observable

We have 5 types of Observables accompanied by 5 corresponding types of Observers. Each type of Observable is used in different cases based on the quantity and type of elements emitted by the Observable

Observable	Observer	# of emissions
Observable	Observer	Multiple or None
Single	SingleObserver	One
Maybe	MaybeObserver	One or None
Flowable	Observer	Multiple or None
Completable	CompletableObserver	None

RxJava - Observable

- **Just**: Available: Flowable, Observable, Maybe, Single Create an Observable that emits a specific item.
- **Defer**: Available: Flowable, Observable, Maybe, Single, Completable Do not create an Observable until there is an Observer subscribed, and create a new Observable every time a new Observer subscribes.
- **From**: Available: Flowable, Observable Convert other objects and data types into Observables.
- **Interval**: Available: Flowable, Observable Periodically emit an infinite sequence (Long), increasing over time.
- **FromCallable**: Available: Flowable, Observable, Maybe, Single, Completable When an observer subscribes, the provided Callable is invoked, and its return value (or exception) is forwarded to the Observer.
- **Create**: Available: Flowable, Observable, Maybe, Single, Completable Create an Observable that can emit data during processing by calling onNext() to the Observer

RxJava - Observer

- An Observer in RxJava is typically implemented using an interface or an abstract class with four main methods:
- **onSubscribe(Disposable d)**: This method is called when the Observer is subscribed to an Observable. Through the "Disposable" object, you can use it to unsubscribe and stop observing the Observable if needed.
- **onNext(T t)**: This method is called when the Observable emits a data item (item). The data value is passed as the parameter "t."
- **onError(Throwable e)**: This method is called when an error occurs during the processing of data by the Observable. The error is passed as a "Throwable" object.
- **onComplete()**: This method is called when the Observable completes the data emission process it has performed. After calling this method, the Observable no longer emits data.

Rxjava : Manage concurrency

- Schedulers is an essential component used to manage and determine the scheduler on which elements in a data stream will be processed. Schedulers help you control whether data should be processed on threads such as background threads, UI threads, or any other thread that suits a specific taks

```
observable
        .subscribeOn(Schedulers.io()) // Pr
        .observeOn(Schedulers.newThread())
```

Rxjava : Disposable

- In RxJava, a Disposable is an object used to manage the unsubscription of an Observer from an Observable. When an Observer subscribes to observe an Observable, a Disposable is returned, allowing you to control the cessation of observation and the release of resources when no longer needed

Rxjava :

- Demo

Section 3

RxKotlin

RxKotlin

- RxKotlin is a lightweight library, serving as extension functions for RxJava. RxKotlin is designed with the purpose of optimizing and streamlining the use of RxJava in conjunction with Kotlin
- Please note that RxKotlin does not utilize coroutines. The reason is quite straightforward: both coroutines and Schedulers in RxKotlin operate in a similar manner. While coroutines are relatively new, Schedulers have been around for a long time alongside RxJava, RxJs, RxSwift, and others.
- Coroutines are suitable for developing concurrent applications in scenarios where we cannot use RxKotlin Schedulers

RxKotlin

- RxKotlin is an extension library for RxJava, it doesn't introduce many features that RxJava doesn't have. Instead, RxKotlin provides some more readable syntax for using RxJava in Kotlin. Here are some examples of the syntactic features that RxKotlin offers

RxKotlin

- RxKotlin primarily provides syntax and utilities to make using RxJava in Kotlin more convenient. Here are some additional points about RxKotlin:
- **Operators and Extension Functions**: RxKotlin offers a range of operators and extension functions for RxJava to perform operations on Observables and Flowables. This helps in writing concise and readable code in Kotlin.
- **Interop with Kotlin Coroutine**: RxKotlin allows for easy integration of RxJava and Kotlin Coroutine. This enables more flexible handling of asynchronous tasks using coroutine syntax.
- **Support for Collections**: RxKotlin provides extension methods to convert collections and lists into Observables or Flowables, making it easier to work with data from these data structures.
- **Kotlin-Friendly**: RxKotlin is designed with support for the Kotlin language, utilizing nullable types and smart casts to reduce errors and make the source code more readable when using RxJava.
- **Lifecycle Management**: While RxKotlin doesn't provide built-in lifecycle awareness like LiveData in Android, you can use extension functions from other libraries such as RxLifecycle to help manage the connections of Observables with the Android component lifecycle.

RxKotlin

RxJava:

kotlin Copy code

```kotlin
import io.reactivex.Observable
import io.reactivex.schedulers.Schedulers

fun main() {
    val observable = Observable.just(1, 2, 3, 4, 5)

    observable
        .observeOn(Schedulers.io())
        .map { it * 2 }
        .subscribe { value -> println(value) }

    Thread.sleep(1000)
}
```

RxKotlin:

kotlin Copy code

```kotlin
import io.reactivex.rxkotlin.toObservable
import io.reactivex.schedulers.Schedulers

fun main() {
    val observable = listOf(1, 2, 3, 4, 5).toObservable()

    observable
        .observeOn(Schedulers.io())
        .map { it * 2 }
        .subscribe { value -> println(value) }

    Thread.sleep(1000)
}
```

RxKotlin

RxJava:

kotlin Copy code

```kotlin
import io.reactivex.Observable

fun main() {
    val observable = Observable.just(1, 2, 3, 4, 5)

    observable.subscribe(
        { value -> println("Received: $value") },
        { error -> println("Error: ${error.message}") },
        { println("Completed") }
    )

    Thread.sleep(1000)
}
```

RxKotlin:

kotlin Copy code

```kotlin
import io.reactivex.rxkotlin.subscribeBy

fun main() {
    val observable = listOf(1, 2, 3, 4, 5).toObservable()

    observable.subscribeBy(
        onNext = { value -> println("Received: $value") },
        onError = { error -> println("Error: ${error.message}") },
        onComplete = { println("Completed") }
    )

    Thread.sleep(1000)
}
```

Section 4

Common operator usuage

Overview operator

In RxJava, an "**operator**" is a crucial component for processing and transforming data within Observable streams. An operator is a function or method used to perform transformations, filtering, combining, or applying other operations to the data within an Observable, resulting in a new Observable.

- Filtering/suppressing operators
- Transforming operators
- Reducing operators
- Error handling operators
- Utility operators

Filtering/suppressing operators

These types of operators help us filter out elements emitted that do not meet certain criteria, thereby preventing these elements from being sent to the Observer:

- **filter()**
- **take()**
- **skip()**
- **takeWhile() và skipWhile():**
- **distinct()**
- **distinctUntilChanged()**

```
import io.reactivex.Observable;
public class Launcher {
 public static void main(String[] args) {
 Observable.just("Alpha", "Beta", "Gamma", "Delta", "Epsilon")
 .filter(s -> s.length() != 5)
 subscribe(s -> System.out.println("Nhan: " + s));
 }
}
```

Transforming operators

This type of operator helps us transform and modify elements emitted before they are delivered to the Observer

- **map()**
- **cast()**
- **sorted()**
- **delay()**

Collection operators

The functions within the Collection category help us accumulate all emitted elements into a collection, such as a list or a map, and then emit the entire collection in a single emission. Let's go through some representative functions commonly used in this type of operator

- **toList()**
- **toSortedList()**
- **sorted()**
- **delay()**

Error recovery operators

In case you want to handle errors before they reach the Observer or handle errors instead of stopping the program, you can use the following operators, or you can handle them in your own way.

- **onErrorReturn() và onErrorReturnItem()**
- **onErrorResumeNext()**
- **retry()**

Action operators

operators that aid in the debugging process and allow us to observe the functioning of an Observable chain through `doOn` operators

- **doOnNext(), doOnComplete(), and doOnError()**
- **doOnSubscribe() and doOnDispose()**
- **doOnSuccess()**

THANK YOU!

DuyTD13